Что задают на собеседовании:

По сишарпу спрашивают базовые вещи, например Делегат, синтаксис объявления делегатов, стандартные делегаты в сишарпе

По Гиту спрашивают Что такое git revert, git cherry-pick, force push

Про паттерны проектирования спрашивали меня, лучше штуки 3 знать, чтобы было что сказать

Значимые типы, ссылочные типы и тд

Спрашивали какие интерфейсы стандартные знаешь

И про INotifyPropertyChanged вроде спрашивали, ICommand

Про mvvm да, спрашивали но не глубоко

Отличия event от delegate

Какие знаешь коллекции в сишарпе тд

1. Поля

Поля экземпляра — это переменные, которые принадлежат каждому отдельному объекту класса или структуры. У каждого объекта своё собственное значение такого поля.

Статические поля принадлежат самому типу, а не объекту. Такое поле существует в единственном экземпляре для всех объектов этого типа, и его значение общее для всех.

Доступ к статическому полю нужно получать через имя типа, а не через объект. Если попытаться обратиться к статическому полю через объект, будет ошибка компиляции.

Обычно поля класса делают **private** или **protected**, чтобы ограничить к ним прямой доступ извне и защитить внутренние данные от неправильного использования. Для работы с такими данными используют **методы, свойства или индексаторы** — это позволяет контролировать, какие значения можно устанавливать, и не допускать ошибок. **Поле**, в котором хранится значение, **доступное через открытое свойство**, называют резервным полем или **резервным хранилищем**.

---

2. Свойства

Свойства представляют собой что-то вроде «ложных полей». Внешне работа с ними выглядит как работа с обычными полями.

Свойства — это специальные методы в классе, которые позволяют безопасно читать и изменять значения полей. Через свойства можно контролировать доступ к данным: например, проверять вводимые значения или делать только чтение. Для пользователя свойства выглядят как обычные переменные, но внутри них можно добавить любую логику.

Блок **get** обязательно должен иметь в себе возврат значения того же типа, что и свойство, а в блоке **set** значение, которое было присвоено свойству, можно получить при помощи ключевого слово **value**. **Блока set может и не быть**, если свойство не поддерживает запись и доступно только для чтения.

---

3. Абстрактные классы

Ключевое слово **abstract** позволяет создавать классы и члены классов, которые являются неполными и должны быть **реализованы в производном классе**.

Создавать экземпляры абстрактного класса нельзя. Назначение абстрактного класса заключается в предоставлении общего определения для базового класса, которое могут совместно использовать несколько производных классов.

---

4. virtual

Ключевое слово **virtual** используется для создания виртуальных методов — таких функций в базовом классе, которые можно переопределить в производных классах.

Разница между virtual и abstract методами в C# заключается в следующем:

- **Virtual (виртуальный) метод**
  - Может иметь реализацию (тело) в базовом классе.
  - Его можно переопределить в производном классе, но не обязательно.
  - Позволяет изменить поведение метода при необходимости, сохраняя при этом базовую реализацию.
- **Abstract (абстрактный) метод**
  - Не имеет реализации в базовом классе (тело отсутствует).
  - Объявляется только в абстрактном классе.
  - Обязателен к переопределению в каждом производном классе.
  - Используется, чтобы заставить все наследники реализовать этот метод по-своему.

Иными словами, абстрактный метод — это виртуальный метод без реализации, который обязательно нужно реализовать в наследниках, а виртуальный метод — это метод с реализацией, которую можно, но не обязательно нужно переопределять.

---

5. Запечатанные классы

Запечатанные классы и методы в C# — это конструкции, которые запрещают наследование или переопределение.

- **Запечатанный класс (sealed class)** нельзя использовать как базовый класс — от него нельзя наследоваться. Это защищает класс от изменений через наследование и может ускорить выполнение программ.
- **Запечатанный метод или свойство** — это переопределённый виртуальный член класса, который нельзя переопределять в дальнейших наследниках. Для этого перед ключевым словом `override` ставят `sealed`. Так метод перестаёт быть виртуальным для потомков.

Итог: `sealed` ограничивает расширяемость, защищая класс или метод от наследования и переопределения.

В следующем примере класс B наследуется от класса A, но не может наследоваться от класса B.

```
class A {}
sealed class B : A {}
```

В следующем примере Z наследуется от Y, но Z не может переопределить виртуальную функцию F, объявленную в X и запечатанную в Y.

```
class X
{
```

```
    protected virtual void F() { Console.WriteLine("X.F"); }
    protected virtual void F2() { Console.WriteLine("X.F2"); }
}

class Y : X
{
    sealed protected override void F() { Console.WriteLine("Y.F"); }
    protected override void F2() { Console.WriteLine("Y.F2"); }
}

class Z : Y
{
    // Attempting to override F causes compiler error CS0239.
    // protected override void F() { Console.WriteLine("Z.F"); }

    // Overriding F2 is allowed.
    protected override void F2() { Console.WriteLine("Z.F2"); }
}
```

---

6. Интерфейсы (`INotifyPropertyChanged`, `ICommand`)

Для поддержки множественного наследования в языке C# и ряде других ООП-языках есть так называемые интерфейсы
Интерфейс в C# — это своего рода договор (контракт), который определяет набор методов, свойств и других членов без их реализации. Класс или структура, реализующие этот интерфейс, обязаны реализовать все его члены.
Они, во многом, похожи на абстрактные классы (наследование от классов ≈ реализация интерфейсов):

- они являются типами, экземпляры которых можно создать, только создав экземпляр класса, реализующего их;
- при реализации интерфейсов необходимо определять все их члены.

Однако у интерфейсов есть ряд особенностей:
один класс может реализовывать сразу несколько интерфейсов;
структуры также могут реализовывать интерфейсы;
в интерфейсах нельзя определять поля;
в интерфейсах нельзя реализовать методы и свойства, только определять, то есть все методы и свойства абстрактные (в новых версиях C# можно, но тогда это будет не совсем интерфейс).

Стандартные интерфейсы

- **IEnumerable и IEnumerator** — позволяют объектам поддерживать перебор элементов (например, в цикле foreach). Это основа для коллекций и удобной работы с ними.
- **ICollection и IList** — расширяют возможности коллекций, добавляя методы для добавления, удаления и доступа по индексу.
- **IDisposable** — интерфейс для освобождения ресурсов (например, закрытие файлов или соединений). Класс, реализующий IDisposable, должен реализовать метод Dispose().
- **IComparable и IComparer** — используются для сравнения объектов, например, при сортировке.
- **INotifyPropertyChanged** — интерфейс для уведомления об изменении свойств объекта, часто применяется в MVVM для обновления UI.

- **ICommand** — описывает команду с методами для выполнения и проверки возможности выполнения, широко используется в паттерне MVVM.

- `IQueryable` – для работы с LINQ.

Интерфейсы помогают задавать поведение для разных классов, обеспечивают гибкость и позволяют писать более расширяемый и поддерживаемый код.

1. Пример интерфейса IEnumerable (перебор коллекции)

```
class MyCollection : IEnumerable
{
    private int[] numbers = { 1, 2, 3, 4, 5 };

    public IEnumerator GetEnumerator()
    {
        for (int i = 0; i < numbers.Length; i++)
        {
            yield return numbers[i];
        }
    }
}

class Program
{
    static void Main()
    {
        var collection = new MyCollection();
        foreach (var item in collection)
        {
            Console.WriteLine(item);
        }
    }
}
```

Объяснение:

Класс реализует IEnumerable, чтобы его можно было перебирать в цикле foreach.

2. Пример интерфейса IDisposable (освобождение ресурсов)

```
class ResourceHolder : IDisposable
{
    private bool disposed = false;

    public void UseResource()
    {
        if (disposed) throw new ObjectDisposedException("ResourceHolder");
        Console.WriteLine("Использование ресурса");
    }
```

```
    public void Dispose()
    {
        if (!disposed)
        {
            Console.WriteLine("Освобождение ресурсов");
            disposed = true;
        }
    }
}

class Program
{
    static void Main()
    {
        using (var resource = new ResourceHolder())
        {
            resource.UseResource();
        } // Здесь автоматически вызывается Dispose()
    }
}
```

Объяснение:

Интерфейс IDisposable позволяет освободить ресурсы, например, файлы или соединения, когда объект больше не нужен.

3. Пример интерфейса IComparable (сравнение объектов для сортировки)

```
class Person : IComparable<Person>
{
    public string Name { get; set; }
    public int Age { get; set; }

    public int CompareTo(Person other)
    {
        return this.Age.CompareTo(other.Age);
    }
}


class Program
{
    static void Main()
    {
        var people = new Person[]
        {
            new Person { Name = "Аня", Age = 30 },
            new Person { Name = "Борис", Age = 25 },
            new Person { Name = "Вика", Age = 35 }
        };

        Array.Sort(people);
```

```
        foreach (var person in people)
        {
            Console.WriteLine($"{person.Name}, {person.Age}");
        }
    }
}
```

Объяснение:
IComparable позволяет задать правило сравнения объектов, чтобы их можно было сортировать.

`INotifyPropertyChanged`

Классы модели представления в WPF должны реализовывать интерфейс INotifyPropertyChanged («оповестить об изменении события»). Он находится в пространстве имен System.ComponentModel

Это интерфейс в C#, который используется для уведомления, что значение свойства объекта изменилось. Используется чтобы автоматически обновлять интерфейс пользователя при изменении данных.

Когда свойство меняется, класс вызывает событие **PropertyChanged**, и система обновляет все связанные элементы интерфейса. Чтобы это работало, класс реализует INotifyPropertyChanged и в сеттере свойства вызывает это событие.

Нужен, чтобы **View (окно) узнавало об изменениях** в `ViewModel`.

```
public class UserViewModel : INotifyPropertyChanged
{
    private string _name;
    public string Name
    {
        get => _name;
        set
        {
            _name = value;
            OnPropertyChanged(); // Говорим View: "Name изменился!"
        }
    }

    public event PropertyChangedEventHandler? PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string prop = "")
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(prop));
}
```

Первым параметром при вызове события передается объект, из которого оно исходит (обычно сюда нужно передать ссылку на текущий объект — this).

Вторым параметром передается экземпляр класса PropertyChangedEventArgs («аргументы события изменения свойства»). В их конструктор, в свою очередь, передается строка, содержащая название измененного свойства, или null, если нужно оповестить об изменении сразу всех свойств.

```
OnPropertyChanged(nameof(Foo));
```

Обратите внимание на конструкцию nameof. Компилятор заменяет ее на строковое название члена класса, указанного в круглых скобках. Она нужна для того, чтобы, например, можно было переименовать свойство в Visual Studio.

ICommand

ICommand – интерфейс для команд, которые можно выполнить, например, при нажатии кнопки в пользовательском интерфейсе.

Основные части ICommand:

- Метод **Execute(object parameter)** — выполняет команду.
- Метод **CanExecute(object parameter)** — возвращает, можно ли сейчас выполнить команду (например, кнопка будет активна или нет).
- Событие **CanExecuteChanged** — уведомляет, что состояние команды изменилось (например, кнопка должна стать активной или неактивной).

Реализуя ICommand, вы создаёте **объекты-команды**, которые **можно привязать к элементам интерфейса** (кнопкам, меню и т.п.). Это позволяет управлять поведением UI через логику в модели или представлении, без прямой обработки событий в коде формы.

```
public class RelayCommand : ICommand
{
    private Action<object> _execute;
    public RelayCommand(Action<object> execute) => _execute = execute;

    public bool CanExecute(object? parameter) => true; // Можно ли выполнить
команду?
    public void Execute(object? parameter) => _execute(parameter!);

    public event EventHandler? CanExecuteChanged;
}

// В ViewModel:
public ICommand ClickCommand => new RelayCommand(_ => Console.WriteLine("Кноп
ка нажата!"));
```

---

7. Делегаты

Что такое делегат?

Делегат – это переменная, в которой хранится функция, которую можно вызвать.

Делегат – это тип, который представляет ссылку на метод с определенной сигнатурой. Позволяет передавать методы как аргументы, хранить их в переменных и вызывать.

Делегат – это **"указатель на метод"**. Он позволяет **передавать методы как аргументы**, сохранять их в переменных и вызывать когда нужно.

Пример из жизни:

Представь, что у тебя есть пульт от телевизора (делегат). Ты можешь запрограммировать его на вызов разных функций:

- Кнопка 1 → Включить музыку
- Кнопка 2 → Показать погоду

В коде это выглядит так:

```
// 1. Объявляем делегат (тип метода)
public delegate void RemoteControlAction(string command);

// 2. Методы, которые можно "привязать" к делегату
void PlayMusic(string song) => Console.WriteLine($"Играет: {song}");
void ShowWeather(string city) => Console.WriteLine($"Погода в {city}: 25°C");

// 3. Создаем экземпляр делегата и настраиваем
RemoteControlAction action = PlayMusic; // вешаем метод PlayMusic на делегат
action("Queen - Bohemian Rhapsody"); // Выведет: "Играет: Queen - Bohemian Rh
apsody"

action = ShowWeather; // теперь делегат вызывает другой метод
action("Москва"); // Выведет: "Погода в Москве: 25°C"
```

Стандартные делегаты (`Action`, `Func`, `Predicate`)

В C# есть готовые делегаты, чтобы не объявлять свои:

1. `Action` – для методов **без возвращаемого значения** (void).

```
Action<string> log = message => Console.WriteLine(message);
log("Привет!"); // Выведет: "Привет!"
```

2. `Func` – для методов **с возвращаемым значением**.

```
Func<int, int, int> sum = (a, b) => a + b;
int result = sum(5, 3); // Вернет 8
```

3. `Predicate` – для проверки условия (возвращает `bool`).

```
Predicate<int> isEven = num => num % 2 == 0;
bool check = isEven(4); // Вернет true
```

---

8. Отличия `event` от `delegate`

- **Делегат** – это **тип**, который хранит ссылку на метод.
- **Событие (`event`)** – множество делегатов, которые разом можно вызвать внутри класса.

  Это **надстройка над делегатом**, которая:

  - Позволяет безопасно **добавлять (+=) и убирать (-=) подписчиков**.
  - **Не даёт вызвать событие извне класса** (только внутри).

Пример:

```
public class Button
{
    public delegate void ClickHandler(); // Делегат
    public event ClickHandler? Clicked; // Событие

    public void Press()
    {
        Clicked?.Invoke(); // Вызов события
    }
}

// Где-то в коде:
Button btn = new Button();
btn.Clicked += () => Console.WriteLine("Кнопка нажата!"); // Подписались
btn.Press(); // Выведет: "Кнопка нажата!"
```

---

9. Git (важные команды)

`git revert` **vs** `git reset`

- `git revert` – **безопасная отмена коммита**. Он создаёт **новый коммит**, который отменяет изменения старого.

  ```
  git revert abc123  # отменяет коммит abc123, но не удаляет его из истории
  ```

  → Подходит, если изменения уже были отправлены на сервер.

- `git reset` – **удаляет коммиты** (опасно, если работаешь в команде).

  ```
  git reset --hard abc123  # стирает все коммиты после abc123
  ```

  → Используй только в локальной ветке!

`git cherry-pick`

Позволяет **перенести отдельный коммит** из одной ветки в другую.

```
git cherry-pick abc123  # применяет изменения коммита abc123 в текущую ветку
```

→ Полезно, если нужно **забрать исправление бага** из `dev` в `main`, не сливая всю ветку.

`git push --force` **(force push)**

Перезаписывает удалённую ветку **без проверки конфликтов**.

```
git push --force origin main
```

□□ **Опасно!** Может **удалить чужие коммиты**. Используй только если:

- Ты один работаешь в ветке.
- Надо исправить историю **перед Pull Request**.

---

10. Паттерны проектирования

MVVM (Model-View-ViewModel)

Для чего? Чтобы отделить логику (`ViewModel`) от интерфейса (`View`).

Как работает?

- **View (окно, кнопки)** – только отображает данные.
- **ViewModel** – содержит логику и данные, уведомляет View об изменениях через `INotifyPropertyChanged`.
- **Model** – данные (например, класс `User`).

Пример:

```
// ViewModel
public class UserViewModel : INotifyPropertyChanged
{
    private string _name;
    public string Name
    {
        get => _name;
        set
        {
            _name = value;
            OnPropertyChanged(); // Уведомляем View, что имя изменилось
        }
    }

    public event PropertyChangedEventHandler? PropertyChanged;
    protected void OnPropertyChanged([CallerMemberName] string prop = "")
        => PropertyChanged?.Invoke(this, new PropertyChangedEventArgs(prop));
}
```

Singleton (Одиночка)

Гарантирует, что класс будет иметь **только один экземпляр**.

```
public class Logger
{
```

```
    private static Logger _instance;
    public static Logger Instance => _instance ??= new Logger();

    private Logger() { } // Приватный конструктор

    public void Log(string message) => Console.WriteLine(message);
}

// Использование:
Logger.Instance.Log("Hello!"); // Всегда один и тот же объект
```

Factory Method

Позволяет **создавать объекты без указания точного класса**.

Это порождающий паттерн, который помогает создавать объекты, не указывая конкретный класс создаваемого объекта напрямую. Вместо этого используется специальный метод — фабрика, который решает, какой именно объект создать.

```
// Общий интерфейс продукта
interface ITransport
{
    void Deliver();
}
// Конкретный продукт — грузовик
class Truck : ITransport
{
    public void Deliver()
    {
        Console.WriteLine("Доставка грузовиком по земле");
    }
}
// Конкретный продукт — корабль
class Ship : ITransport
{
    public void Deliver()
    {
        Console.WriteLine("Доставка кораблем по морю");
    }
}
// Абстрактный создатель с фабричным методом
abstract class Logistics
{
    // Фабричный метод — создает объект ITransport
    public abstract ITransport CreateTransport();

    public void PlanDelivery()
    {
        var transport = CreateTransport();
        transport.Deliver();
```

```
    }
}
// Конкретный создатель для наземной логистики
class RoadLogistics : Logistics
{
    public override ITransport CreateTransport()
    {
        return new Truck();
    }
}
// Конкретный создатель для морской логистики
class SeaLogistics : Logistics
{
    public override ITransport CreateTransport()
    {
        return new Ship();
    }
}

class Program
{
    static void Main()
    {
        Logistics logistics;

        logistics = new RoadLogistics();
        logistics.PlanDelivery();  // Выведет: Доставка грузовиком по земле

        logistics = new SeaLogistics();
        logistics.PlanDelivery();  // Выведет: Доставка кораблем по морю
    }
}
```

Объяснение:

- Есть общий интерфейс ITransport с методом Deliver().
- Классы Truck и Ship реализуют этот интерфейс по-разному.
- Абстрактный класс Logistics содержит фабричный метод CreateTransport(), который должен создавать объект ITransport.
- Конкретные классы RoadLogistics и SeaLogistics переопределяют фабричный метод, создавая нужный тип транспорта.
- Клиентский код работает с абстрактным классом Logistics и не зависит от конкретных классов транспорта.

Такой подход позволяет легко добавлять новые виды транспорта, не меняя существующий код

---

11. Значимые vs ссылочные типы

Значимые типы хранят свои данные непосредственно в переменной. При присвоении или передаче значения создаётся копия данных. К ним относятся примитивные типы (int, double, bool), структуры (struct) и перечисления (enum). Обычно они хранятся в стеке, что даёт быстрый доступ и эффективное управление памятью.

Ссылочные типы хранят в переменной не сами данные, а ссылку на объект в куче. При присвоении копируется ссылка, а не объект, поэтому несколько переменных могут указывать на один и тот же объект. К ним относятся классы (class), массивы, строки (string) и делегаты. Память для объектов управляется сборщиком мусора.

Важное отличие: при изменении значимого типа копия меняется отдельно, а при изменении объекта ссылочного типа изменения видны всем, кто ссылается на этот объект.

Также ссылочные типы могут принимать значение null, а значимые — нет (если не использовать Nullable).

```
int a = 5;
int b = a; // копия значения
b = 10;
Console.WriteLine(a); // выведет 5 — a не изменился

class Person { public string Name; }
Person p1 = new Person { Name = "Alice" };
Person p2 = p1; // копия ссылки
p2.Name = "Bob";
Console.WriteLine(p1.Name); // выведет "Bob" — объект один на двоих
```

Boxing / Unboxing

- **Boxing** – преобразование значимого типа в ссылочный (`int` → `object`).
- **Unboxing** – обратное преобразование (`object` → `int`).

```
int num = 42;
object boxed = num; // Boxing (упаковка)
int unboxed = (int)boxed; // Unboxing (распаковка)
```

Критерий	Значимые (`int`, `struct`, `double`)	Ссылочные (`class`, `string`, `delegate`)
Где хранятся?	В стеке (быстро)	В куче (медленнее)
Копирование	Копируется значение	Копируется ссылка

Пример	`int a = 5; int b = a;`
	→ b не влияет на a
		`List<int> list1 = new List<int>();`
		`List<int> list2 = list1;`
		→ `list2` и `list1` указывают на один объект

---

12. Коллекции в C#

Коллекции — это структуры данных для хранения и управления группами объектов. Они удобнее массивов, так как обычно динамически меняют размер.

Коллекция	Описание	Когда использовать?
`List<T>`	Динамический массив однотипных элементов с доступом по индексу	Когда нужен простой список
`Dictionary<TKey, TValue>`	Хранит пары ключ-значение (как словарь). Доступ к элементам по ключу	Быстрый поиск по ключу
`ObservableCollection<T>`	Отличие от List в том, что автоматически уведомляет UI об изменениях. Реализует интерфейс INotifyCollectionChanged , который генерирует события об изменениях коллекции.	В WPF/UWP для привязки данных
`HashSet<T>`	Хранит только уникальные элементы	Проверка уникальности
`Queue<T>`	FIFO (первый вошёл – первый вышел)	Очередь задач
`Stack<T>`	LIFO (последний вошёл – первый вышел)	Отмена действий (Ctrl+Z)

Пример `ObservableCollection` **(для MVVM):**

```
ObservableCollection<string> names = new ObservableCollection<string>();
names.Add("Алиса"); // UI автоматически обновится!
```

Queue<T> (Очередь) — принцип FIFO (First In, First Out)

- Это коллекция, где элементы добавляются в конец и извлекаются из начала.
- Представьте очередь в магазине: первый пришёл — первый обслужен.
- Используется, когда важен порядок обработки элементов, например, задачи на выполнение по очереди.

Основные методы:

- Enqueue(item) — добавить элемент в очередь.
- Dequeue() — взять и удалить первый элемент из очереди.
- Peek() — посмотреть первый элемент, не удаляя его.

```
Queue<string> queue = new Queue<string>();

queue.Enqueue("Первый");
queue.Enqueue("Второй");
queue.Enqueue("Третий");

Console.WriteLine(queue.Dequeue()); // Выведет "Первый"
Console.WriteLine(queue.Peek());    // Выведет "Второй" (следующий в очереди)
Console.WriteLine(queue.Dequeue()); // Выведет "Второй"
```

Stack<T> (Стек) — принцип LIFO (Last In, First Out)

- Это коллекция, где элементы добавляются и извлекаются с одного конца — вершины стека.
- Представьте стопку тарелок: последняя положенная сверху — первая снятая.

- Используется, когда нужно работать с последним добавленным элементом, например, отмена действий (undo).

Основные методы:

- Push(item) — положить элемент на вершину стека.
- Pop() — снять и вернуть верхний элемент стека.
- Peek() — посмотреть верхний элемент, не снимая его.

```
Stack<string> stack = new Stack<string>();

stack.Push("Первый");
stack.Push("Второй");
stack.Push("Третий");

Console.WriteLine(stack.Pop());  // Выведет "Третий"
Console.WriteLine(stack.Peek()); // Выведет "Второй" (следующий сверху)
Console.WriteLine(stack.Pop());  // Выведет "Второй"
```

13. Модификаторы доступа

Это специальные ключевые слова, которые определяют, из каких частей программы можно обращаться к классам, методам, полям и другим членам. Они помогают контролировать видимость и защищают данные, реализуя принцип инкапсуляции.

Основные модификаторы доступа в C#:

- **private**
  Доступен только внутри того же класса или структуры. Самый закрытый уровень доступа. Используется для скрытия деталей реализации.
- **protected**
  Доступен внутри класса и во всех его наследниках (даже если они в других сборках). Позволяет наследникам работать с защищёнными членами.
- **internal**
  Доступен во всей текущей сборке (проекте), но не доступен из других сборок. Используется для ограничения доступа внутри одного приложения или библиотеки.
- **protected internal**
  Комбинация: доступен либо из текущей сборки, либо из производных классов в других сборках.
- **private protected**
  Доступен только внутри класса и производных классов, но только в пределах текущей сборки.
- **public**
  Доступен из любого места — самый открытый уровень доступа.
- **file** (C# 11)
  Доступен только в пределах текущего исходного файла.

14. Модификаторы доступа

Объектно-ориентированное программирование (ООП) — это способ писать программы, представляя всё в виде объектов, похожих на реальные вещи из жизни. Каждый объект — это «коробочка», в которой хранятся данные (свойства) и действия с этими данными (методы).

Принципы ООП:

- **Инкапсуляция** — прятать внутренние детали объекта, чтобы с ним можно было работать только через понятные методы. Внешний код взаимодействует с объектом только через публичные методы и свойства. Это помогает избежать неправильного использования и ошибок.
- **Наследование** — Позволяет создавать новые классы на основе уже существующих, наследуя их свойства и методы. Это облегчает повторное использование кода и создание иерархий объектов. Например, «Гоночная машина» наследует свойства обычной машины, но может ехать быстрее.
- **Полиморфизм** — Даёт возможность использовать один и тот же интерфейс для работы с разными типами объектов. Например, один и тот же метод может вести себя по-разному в зависимости от того, к какому классу принадлежит объект.
  Когда разные объекты могут реагировать на одинаковые команды по-разному. Например, команда «звучать» заставит машину издать гудок, а собаку — лай.
- **Абстракция** – это когда мы выделяем только важные характеристики и действия, скрывая детали. Позволяет работать с объектом через минимальный набор свойств и методов, скрывая сложность реализации.

```
// Класс – шаблон для объектов
class Animal
{
    public string Name { get; set; }

    public virtual void Speak()
    {
        Console.WriteLine("Животное издаёт звук");
    }
}

// Наследование и полиморфизм
class Dog : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Собака лает");
    }
}

class Cat : Animal
{
    public override void Speak()
    {
        Console.WriteLine("Кошка мяукает");
```

```
    }
}

class Program
{
    static void Main()
    {
        Animal myDog = new Dog { Name = "Бобик" };
        Animal myCat = new Cat { Name = "Мурка" };

        myDog.Speak(); // Собака лает
        myCat.Speak(); // Кошка мяукает
    }
}
```

Пример абстракции:

```
// Абстрактный класс — общий шаблон для всех фигур
abstract class Shape
{
    public string Color { get; set; }

    public Shape(string color)
    {
        Color = color;
    }

    // Абстрактный метод — конкретные фигуры должны реализовать сами
    public abstract double GetArea();

    // Абстрактный метод — конкретные фигуры должны реализовать сами
    public abstract double GetPerimeter();

    // Общий метод, который используют все фигуры
    public void PrintInfo()
    {
        Console.WriteLine($"Цвет: {Color}");
        Console.WriteLine($"Площадь: {GetArea()}");
        Console.WriteLine($"Периметр: {GetPerimeter()}");
    }
}

// Конкретная фигура — круг
class Circle : Shape
{
    public double Radius { get; set; }

    public Circle(string color, double radius) : base(color)
    {
        Radius = radius;
```

```
    }

    public override double GetArea()
    {
        return Math.PI * Radius * Radius;
    }

    public override double GetPerimeter()
    {
        return 2 * Math.PI * Radius;
    }
}

// Конкретная фигура — прямоугольник
class Rectangle : Shape
{
    public double Length { get; set; }
    public double Width { get; set; }

    public Rectangle(string color, double length, double width) : base(color)
    {
        Length = length;
        Width = width;
    }

    public override double GetArea()
    {
        return Length * Width;
    }

    public override double GetPerimeter()
    {
        return 2 * (Length + Width);
    }
}

// Использование
class Program
{
    static void Main()
    {
        Shape circle = new Circle("Красный", 5);
        circle.PrintInfo();

        Console.WriteLine();

        Shape rectangle = new Rectangle("Синий", 4, 6);
        rectangle.PrintInfo();
    }
}
```

Объяснение простыми словами:

- В примере Shape — абстрактный класс, который описывает общие свойства и методы для всех фигур, но не знает, как именно считать площадь или периметр.
- Конкретные фигуры (Circle, Rectangle) реализуют эти методы по-своему.
- Благодаря этому можно работать с разными фигурами через один общий тип Shape, не заботясь о деталях их реализации.